Homework 2

Information

Name	UNI
Zhewen Guo	zg2567

Goals

Homework:

1. Plot the raw EEG signal and inspect the signal quality over time. Zoom in on the signal, find the artifacts, and learn to distinguish noisy signals from good quality data. You don't need to submit this part, but it is very important to be able to recognize artifacts if you wish to work with EEG signals going forward.

2. Generate an envelope detection function in MATLAB. The function should use the following format:

   ```
   function [envelope]= envelope_detection(raw_EEG_signal, band_low_cut_off,
   band_high_cut_off, low_cut_off)
   ```

   in which, `band_low_cut_off` and `band_high_cut_off` are *low cut off* and *high cut off* frequencies of *bandpass* filter respectively and, `low_cut_off` is *cut off* frequency of the *lowpass* filter in the model.

   You are free to use any toolbox or function you want, or you may replicate the functions you used in Simulink.

3. Plot the output of your function and the output of the Simulink model (recorded in the lab) on the same figure using different colors. Choose a time window that ensures that the signal is clearly observable. Include two figures:
   1. One figure illustrating a stretch of time where the two outputs are very similar
   2. One figure for a period where the outputs are different

Figure 1. Screenshot of Requirements

Task 2: Generate an envelope detection function in MATLAB.

According to the process during the lab (Figure 2), the raw signals should be filtered with a band-pass filter to capture waves at a target frequency (i.e., 8-12 Hz for alpha component). Then an Absolute Value block was utilized as a rectifier. After that, we used a low-pass filter and Mean block to get the average envelope of the signals.

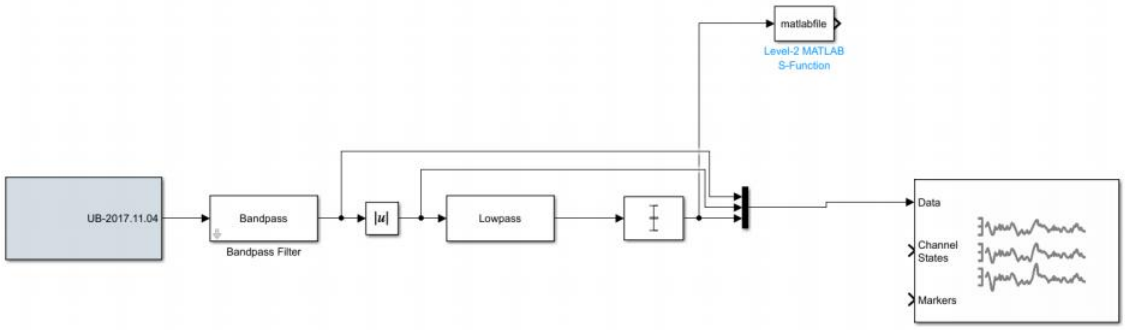


Figure 2. Data process in Simulink

To achieve a similar result, two Butterworth filters are applied to extract alpha signals and envelopes of signals. (Refer to Appendix 1 for full code).

Task 3: Plot the output of both your function and Simulink model.

To calculate the most similar and the most distinct part of two outputs, a sliding window (length = 500, step = 1) is created to calculate the sum of differences (Refer to Appendix 1 for full code).

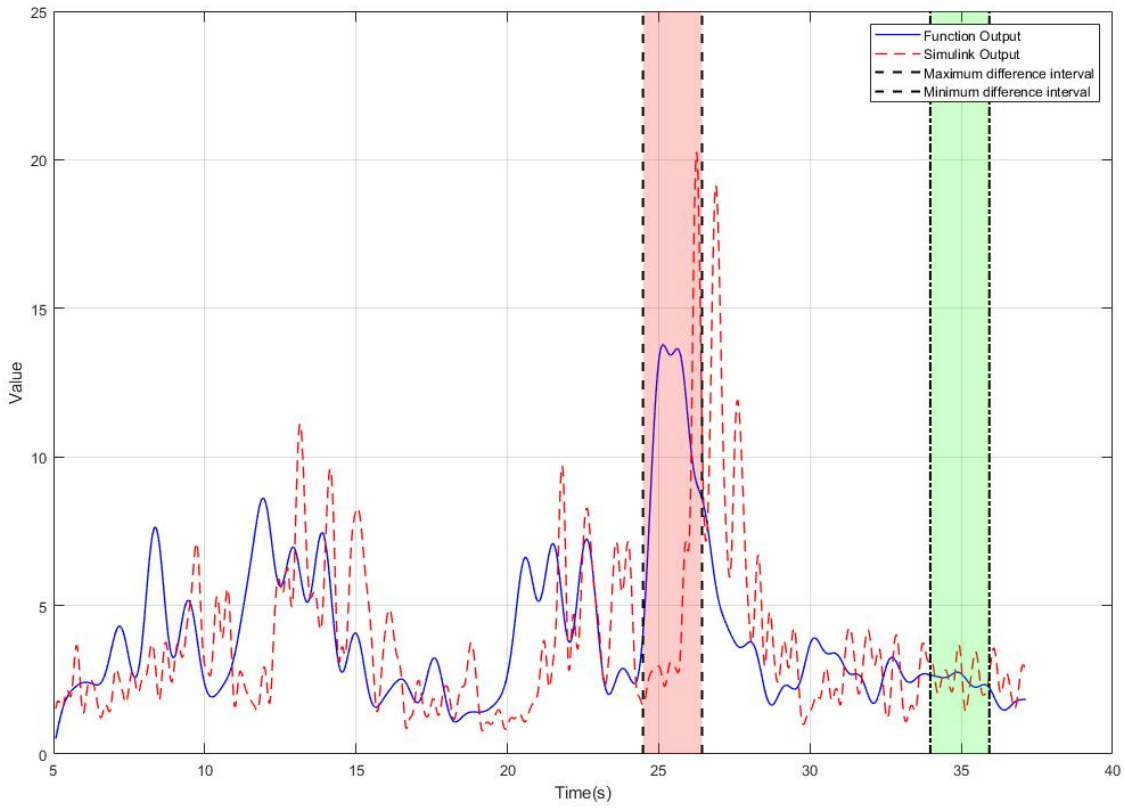


Figure 3. Exhibition of Function Output and Simulink Output

According to the Figure 3, the blue line shows the function output, while the red line demonstrates the result of Simulink. Besides, the red bar covers the range where two outputs reach the most significant difference and the signals in green bar have the highest similarity.

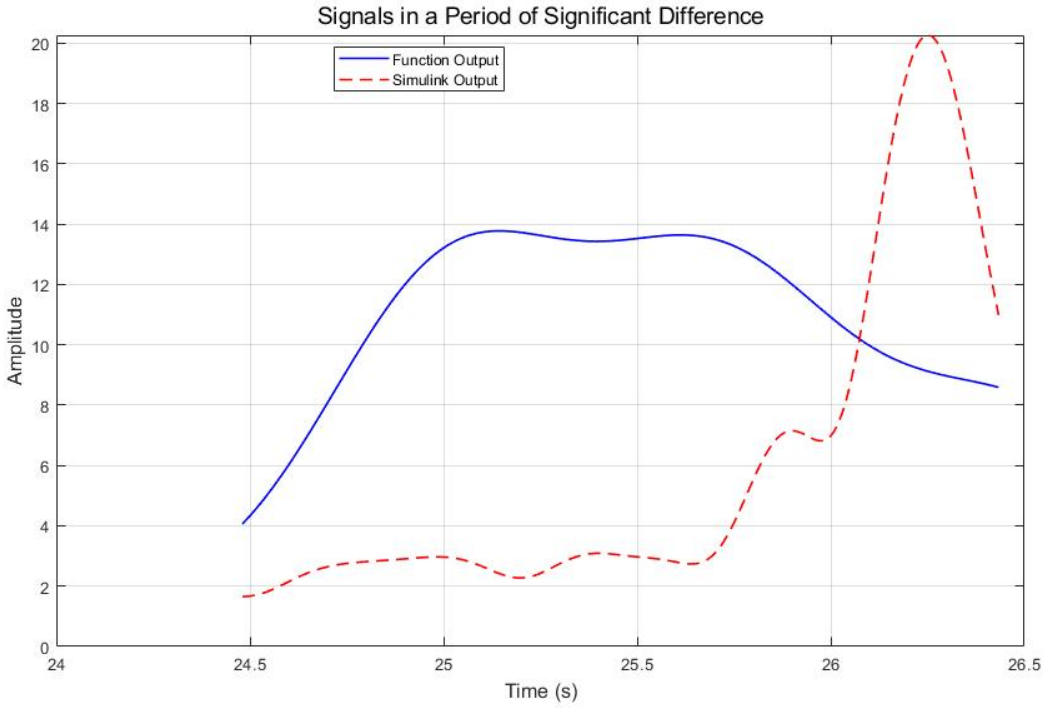


Figure 4. Signals in a Period of Significant Difference

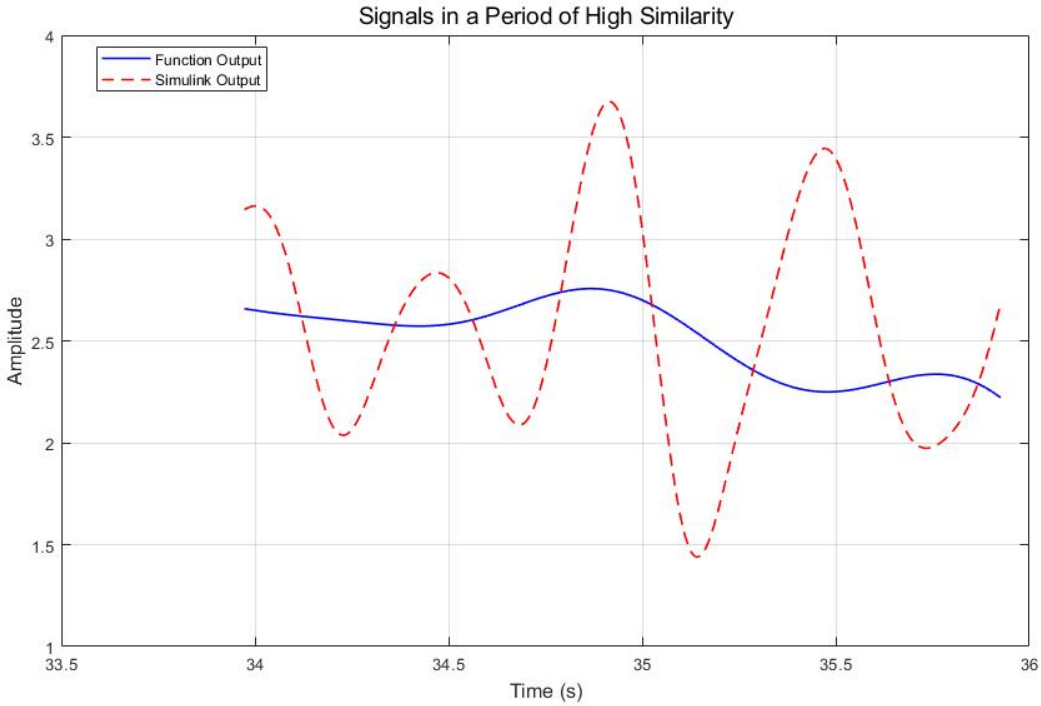


Figure 5. Signals in a Period of High Similarity

However there seems to be an evident phase delay between these two outputs that the processed signal of Simulink is slightly slower, which may lead to a error conclusion. Thus, I tried to manually align the two data sets by delaying the Simulink data sets by 350 sample points, which is approximate 1.36 second. Figure 6-8 display the result.

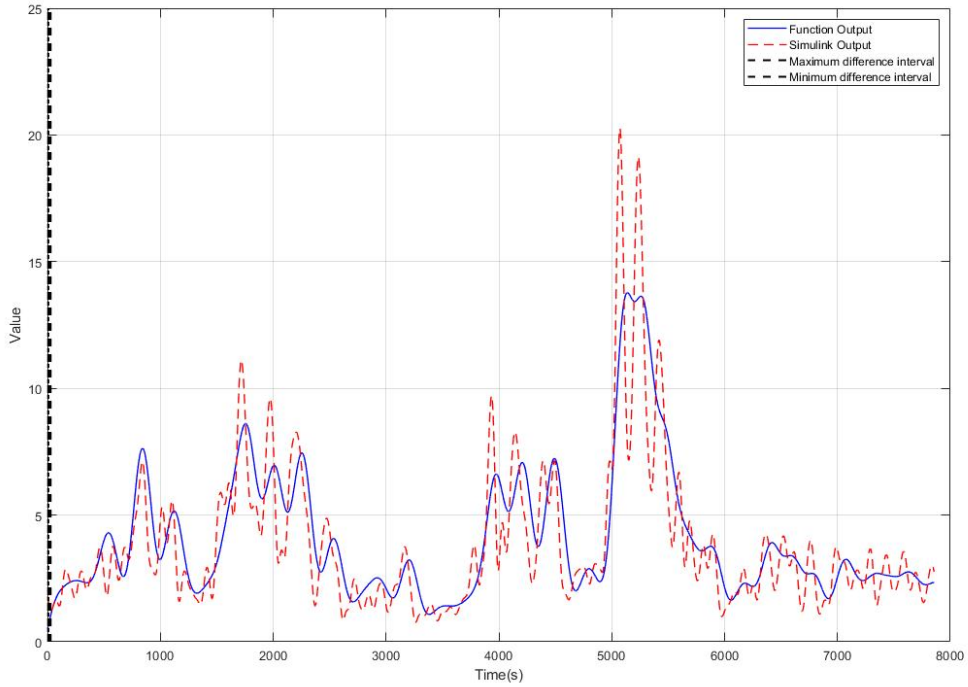


Figure 6. Exhibition of Function Output and Simulink Output After Alignment

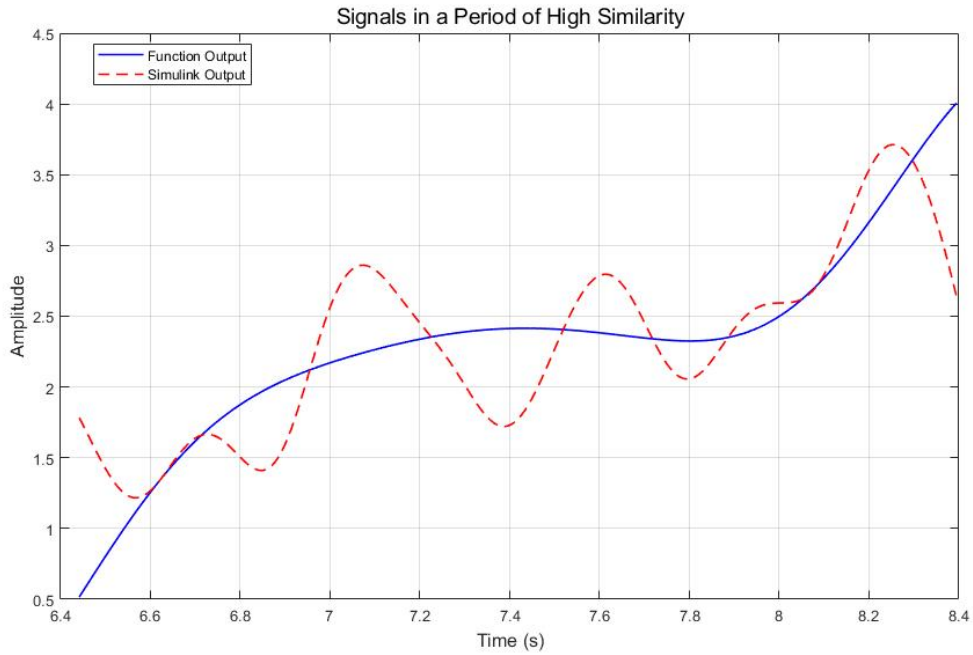


Figure 8. Signals in a Period of High Similarity After Alignment

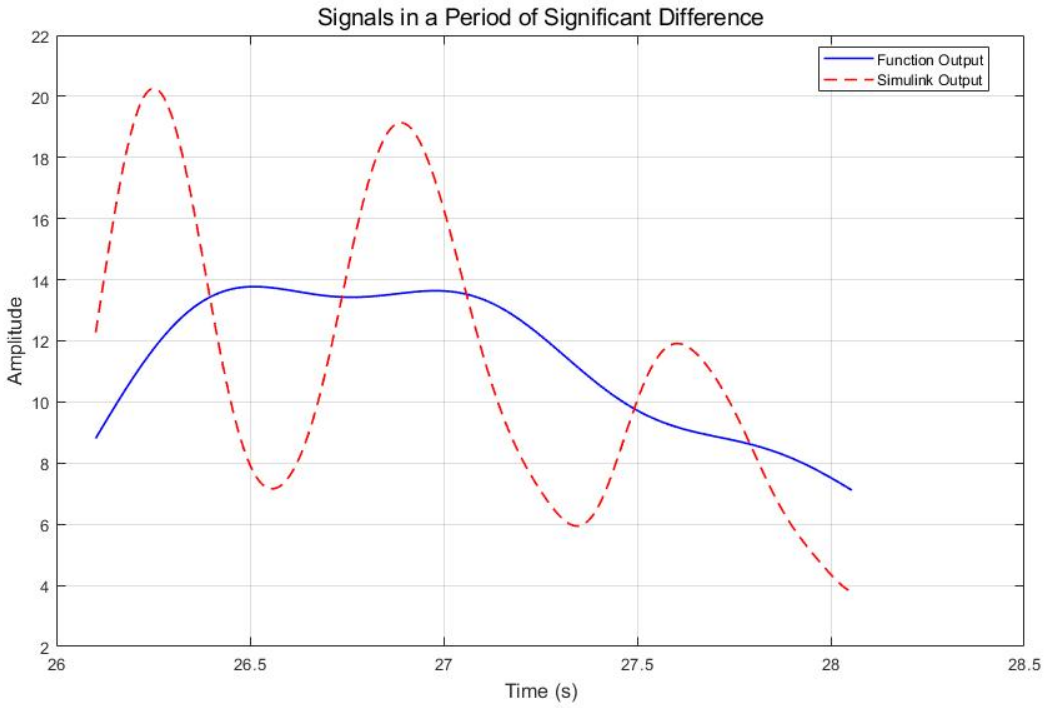


Figure 9. Signals in a Period of Significant Difference After Alignment

Appendix 1:

```
function [envelope] = envelope_detection(raw_EEG_signal, band_low_cut_off,
band_high_cut_off, low_cut_off)
Fs = 256;
N_bandpass = 4;
N_lowpass = 4;
signal_length = length(raw_EEG_signal);
y_time = (0:signal_length-1) / Fs;

% bandpass
[b_band, a_band] = butter(N_bandpass, [band_low_cut_off/(Fs/2),
band_high_cut_off/(Fs/2)], 'bandpass');
% filterd signal
filtered_signal = filtfilt(b_band, a_band, raw_EEG_signal);
%analytic_signal = hilbert(filtered_signal);

%raw_envelope = abs(imag(analytic_signal));
raw_envelope = abs(filtered_signal);
[b_low, a_low] = butter(N_lowpass, low_cut_off/(Fs/2), 'low');

envelope = filtfilt(b_low, a_low, raw_envelope);

end
```

Appendix 2:

```
clc;clear;

Fs = 256;
raw_data = load('F:/Desktop/4090-BCI/HW/HW_2_zg2567/Data/raw_data.mat');
lab_data = load('F:/Desktop/4090-BCI/HW/HW_2_zg2567/Data/mean_data.mat');
addpath('F:/Desktop/4090-BCI/HW/HW_2_zg2567/Code')

time_sample = raw_data.y(1:1, 1300:end);

raw_signal = raw_data.y(2:6, 1300:end);
lab_signal = lab_data.y(2:2,1300:end);

envelopes = zeros(size(raw_signal));
disp(size(time_sample));

for i = 1:5
envelopes(i, :) = envelope_detection(raw_signal(i, :), 8, 13, 1);
end
envelope_mean = mean(envelopes, 1);

function_output = envelope_mean;
simulink_output = lab_signal;


total_length = length(function_output);
t_total = (0:total_length-1) / Fs;
```

```
length_window = 500;
len = length(time_sample);


min_sum = Inf;
min_idx = 1;
max_sum = -Inf;
max_idx = 1;


for i = 1:(len - length_window)
diff_window = abs(function_output(i:i+length_window) -
simulink_output(i:i+length_window));
sum_diff = sum(diff_window);
if sum_diff < min_sum
min_sum = sum_diff;
min_idx = i;
end
if sum_diff > max_sum
max_sum = sum_diff;
max_idx = i;
end
end
fprintf("max_index: %d \n", max_idx);
fprintf("min_index: %d \n", min_idx);
figure('Position', [100, 100, 1200, 800]);
plot(time_sample, function_output, 'b-', 'LineWidth', 1, 'DisplayName', 'Function
Output');
hold on;
plot(time_sample, simulink_output, 'r--', 'LineWidth', 1, 'DisplayName', 'Simulink
Output');

y_lim = ylim;
y_min = y_lim(1);
y_max = y_lim(2);

start_t_max = time_sample(max_idx);
end_t_max = time_sample(max_idx + length_window);
rectangle('Position', [start_t_max, y_min, end_t_max - start_t_max, y_max -
y_min], ...
'FaceColor', 'red', ...
'EdgeColor', 'none', ...
'FaceAlpha', 0.2);

plot([start_t_max, start_t_max], [y_min, y_max], 'k--', 'LineWidth', 1.5);
plot([end_t_max, end_t_max], [y_min, y_max], 'k--', 'LineWidth', 1.5);

start_t_min = time_sample(min_idx);
end_t_min = time_sample(min_idx + length_window);
rectangle('Position', [start_t_min, y_min, end_t_min - start_t_min, y_max -
y_min], ...
'FaceColor', 'green', ...
'EdgeColor', 'none', ...
'FaceAlpha', 0.2);
```

```
plot([start_t_min, start_t_min], [y_min, y_max], 'k-.', 'LineWidth', 1.5);
plot([end_t_min, end_t_min], [y_min, y_max], 'k-.', 'LineWidth', 1.5);

% add label and legend
xlabel('Time(s)');
ylabel('Value');
legend('Function Output', 'Simulink Output', 'Maximum difference interval',
'Minimum difference interval');
grid on;
hold off;

% extract similar data
similar_start_idx = min_idx;
similar_end_idx = min_idx + length_window;
similar_time = time_sample(similar_start_idx:similar_end_idx);
similar_func = function_output(similar_start_idx:similar_end_idx);
similar_simu = simulink_output(similar_start_idx:similar_end_idx);

% extract distinct data
diff_start_idx = max_idx;
diff_end_idx = max_idx + length_window;
diff_time = time_sample(diff_start_idx:diff_end_idx);
diff_func = function_output(diff_start_idx:diff_end_idx);
diff_simu = simulink_output(diff_start_idx:diff_end_idx);

figure('Position', [100, 100, 1000, 600]);
plot(similar_time, similar_func, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Function
Output');
hold on;
plot(similar_time, similar_simu, 'r--', 'LineWidth', 1.2, 'DisplayName', 'Simulink
Output');

title('Signals in a Period of High Similarity', 'FontSize', 14);
xlabel('Time (s)', 'FontSize', 12);
ylabel('Amplitude', 'FontSize', 12);
legend('Location', 'best');
grid on;
box on;
hold off;

figure('Position', [200, 200, 1000, 600]);
plot(diff_time, diff_func, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Function
Output');
hold on;
plot(diff_time, diff_simu, 'r--', 'LineWidth', 1.2, 'DisplayName', 'Simulink
Output');

title('Signals in a Period of Significant Difference', 'FontSize', 14);
xlabel('Time (s)', 'FontSize', 12);
ylabel('Amplitude', 'FontSize', 12);
legend('Location', 'best');
grid on;
box on;
hold off;
```